



Trabalho Acadêmico

Desenvolvimento Web Back-end

Sistema de Gestão de Franquias

WALLACE F G SILVA RU: 5146520
Prof. Rodrigo da S. do Nascimento

Instruções de Submissão e Informações Gerais

Disciplina: Desenvolvimento Web Back-end — Ano 2026

Estudante: WALLACE F G SILVA — **RU:** 5146520

Professor Orientador: Prof. Rodrigo da S. do Nascimento

Repositório Oficial no GitHub:

https://github.com/wallace-pv/Trabalho_Academico_Desenvolvimento_Backend_Gestao_Franquias

Tecnologias: C# 12, ASP.NET Core Web API (.NET 8.0), Entity Framework Core 8.0, SQLite, BCrypt, JWT Bearer, Swagger/OpenAPI.

Guia Rápido de Execução para o Professor Avaliador

A aplicação foi concebida para execução imediata (zero-configuration). O banco de dados relacional SQLite (Franquias.db) é criado e semeado automaticamente no primeiro arranque via DbInitializer.

Comandos para Iniciar a API:

1. git clone
https://github.com/wallace-pv/Trabalho_Academico_Desenvolvimento_Backend_Gestao_Franquias.git
2. cd Trabalho_Academico_Desenvolvimento_Backend_Gestao_Franquias
3. dotnet run --project Franquias.Api
4. Acessar no navegador: **http://localhost:5086/** (redireciona para o Swagger UI interativo)

Usuários e Credenciais Pré-Cadastrados para Teste Imediato

Perfil	E-mail	Senha	Escopo de Acesso
AdminFranqueadora	admin@franquias.com.br	Admin@123	Matriz — Acesso total irrestrito
GestorUnidade	gestor.sp@franquias.com.br	Gestor@123	Gestão Unidade Moema SP
Operador	operador.sp@franquias.com.br	Operador@123	PDV / Caixa Unidade Moema SP
GestorUnidade	gestor.pr@franquias.com.br	Gestor@123	Gestão Unidade Batel Curitiba
Operador	operador.pr@franquias.com.br	Operador@123	PDV / Caixa Unidade Curitiba

1. Descrição do Trabalho e Arquitetura do Sistema

O **Sistema de Gestão de Franquias** é uma API REST corporativa desenvolvida em C# que centraliza a comunicação operacional, logística, contratual e financeira entre a Franqueadora (matriz) e suas Unidades Franqueadas. A solução elimina o uso de planilhas eletrônicas desconexas e fornece endpoints seguros e documentados para administração de usuários com perfis RBAC, catálogo de produtos, fornecedores homologados, movimentações de estoque, registro de vendas, apuração de royalties e indicadores analíticos com LINQ.

Arquitetura em Camadas e Separação de Responsabilidades

A solução adota rigorosa separação de responsabilidades seguindo o padrão de camadas sugerido na disciplina:

- **Controllers (Franquias.Api/Controllers):** Camada de interface HTTP RESTful. Recebe as requisições, valida anotações nos DTOs e retorna os códigos de status HTTP corretos (200, 201, 204, 400, 404, etc.).
- **Services (Franquias.Api/Services):** Camada de regras de negócio. Contém a lógica de verificação de duplicidade de CNPJ, validação de bloqueio de vendas em unidades inativas, controle transacional atômico e cálculo de royalties.
- **Repositories (Franquias.Api/Repositories):** Abstração de acesso a dados (Repository Pattern) implementando consultas otimizadas com Entity Framework Core e LINQ.
- **Data & DbContext (Franquias.Api/Data):** Mapeamento relacional via Fluent API no ApplicationDbContext (chaves primárias, estrangeiras e índices únicos), histórico de Migrations e DbInitializer (seed completo).
- **DTOs (Franquias.Api/DTOs):** Contratos de entrada (Create/Update) e saída (Response) desacoplados do banco de dados, protegendo campos sensíveis como o hash de senhas.
- **Configurations (Franquias.Api/Configurations):** Configuração modular de injeção de dependência, autenticação JWT Bearer, documentação Swagger/OpenAPI e middleware global de exceções.

2. Modelagem Relacional do Banco de Dados

Foi utilizado o **SQLite com Entity Framework Core**, atendendo integralmente ao requisito de banco de dados relacional (chaves primárias, chaves estrangeiras com DeleteBehavior.Restrict/Cascade, índices únicos para integridade e suporte a transações ACID).

Tabela / Entidade	Chaves e Restrições	Descrição no Domínio
Franqueadoras	PK: Id Unique: CNPJ	Matriz da rede, razão social, percentual padrão de royalties.
Responsaveis	PK: Id	Franqueados responsáveis legais por cada unidade franqueada.
Unidades	PK: Id FK: Franqueadora, Responsavel Unique: CNPJ	Lojas físicas com endereço, código único, situação (Ativa/Inativa) e alíquota.
Usuarios	PK: Id FK: UnidadeId Unique: Email	Usuários do sistema com perfil RBAC (Admin, Gestor, Operador) e senha BCrypt.
Categorias	PK: Id	Categorias de produtos (Cafés Especiais, Confeitaria, Workshops, etc.).
Fornecedores	PK: Id Unique: CNPJ	Fornecedores homologados de matérias-primas e insumos.
Produtos	PK: Id FK: Categoria, Fornecedor Unique: SKU	Catálogo de produtos/serviços com preço base e estoque mínimo padrão.
Estoques	PK: Id FK: Unidade, Produto Unique: (Unidade, Produto)	Saldo disponível por produto e unidade física, com alerta de estoque crítico.

MovimentacoesEstoque	PK: Id FK: Unidade, Produto, Usuario	Rastreamento de entradas, saídas por venda e ajustes de inventário.
Vendas	PK: Id FK: Unidade, Usuario Unique: CodigoVenda	Cabeçalho do cupom de venda com total consolidado e data/hora.
ItensVenda	PK: Id FK: Venda, Produto	Itens da venda com snapshot do preço unitário praticado e subtotal.
Royalties	PK: Id FK: Unidade Unique: (Unidade, Mes, Ano)	Cobranças financeiras de royalties baseadas no faturamento consolidado.
Chamados	PK: Id FK: Unidade, UsuarioAbertura	Tickets de suporte e solicitações operacionais entre franquia e matriz.

3. Regras de Negócio Obrigatórias Implementadas

- **Unicidade de CNPJ de Unidades:** Não é permitido cadastrar duas unidades com o mesmo CNPJ. Implementado com `HasIndex(u => u.CNPJ).IsUnique()` no EF Core e validação de serviço (retorna HTTP 400 Bad Request).
- **Unicidade de E-mail de Usuários:** O sistema valida duplicidade de e-mail impedindo contas com o mesmo endereço eletrônico.
- **Bloqueio de Vendas em Unidades Inativas:** Unidades com situação Inativa não podem emitir novas vendas. O `VendaService` rejeita a operação com HTTP 400 descritivo.
- **Composição Obrigatória e Cálculo da Venda:** A venda exige pelo menos um item com quantidade positiva. O valor total é calculado automaticamente a partir dos preços vigentes no catálogo da matriz.
- **Impedimento Estrito de Saldo Negativo de Estoque:** Nenhuma movimentação de saída manual ou conclusão de venda pode deixar o estoque negativo. Se a quantidade demandada for maior que a disponível, a operação é interrompida.
- **Atualização Atômica de Estoque:** A venda utiliza `BeginTransactionAsync` do EF Core, garantindo que o cupom e a baixa nos saldos ocorram atomicamente.
- **Apuração e Cálculo de Royalties:** Calcula automaticamente o valor devido multiplicando o faturamento total das vendas concluídas da unidade na competência pela sua alíquota contratual (ex: 5%), vencendo no dia 10 do mês subsequente.
- **Perfis de Acesso (RBAC):** Autorização via atributos `[Authorize(Roles = '...')]` diferenciando Administrador da Franqueadora, Gestor da Unidade e Operador.
- **Tratamento Global de Exceções:** Middleware centralizado intercepta `BadRequestException` (400), `KeyNotFoundException` (404), `UnauthorizedAccessException` (403) e falhas gerais (500).

4. Resumo das Rotas e Endpoints RESTful Disponíveis

Módulo	Método	Endpoint	Permissão
Autenticação	POST	/api/Auth/login	Anônimo
Autenticação	POST / GET	/api/Auth/registrar /usuarios	Admin / Gestor
Franqueadoras	GET / PUT	/api/Franqueadoras	Autenticado / Admin
Unidades	GET / POST	/api/Unidades	Autenticado / Admin
Unidades	PUT / DELETE	/api/Unidades/{id}	Admin / Gestor
Produtos	GET / POST	/api/Produtos	Autenticado / Admin
Fornecedores	GET / POST	/api/Fornecedores	Autenticado / Admin
Estoques	GET / POST	/api/Estoques/unidade/{id} /movimentar	Autenticado
Vendas	POST / GET	/api/Vendas	Autenticado
Royalties	POST / GET	/api/Royalties/apurar /{id}/pagar	Admin / Gestor
Chamados	POST / PUT	/api/Chamados /{id}/status	Autenticado
Relatórios	GET	/api/Relatorios/ranking-unidades	Admin
Relatórios	GET	/api/Relatorios/dashboard /faturamento	Admin / Autenticado

5. Evidências de Execução dos Testes Automatizados

Apresentação das respostas HTTP reais capturadas durante os testes da API em execução local:

Evidência 1: Autenticação JWT com Retorno de Token e Claims

```
POST http://localhost:5086/api/auth/login
Request Body: {"email": "admin@franquias.com.br", "senha": "Admin@123"}
Status: 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiracao": "2026-09-14T14:47:18Z",
  "usuario": { "id": 1, "nome": "Wallace Silva (Administrador)", "perfil": "AdminFranqueadora" }
}
```

Evidência 2: Bloqueio de Unidade com CNPJ Duplicado (Regra de Negócio)

```
POST http://localhost:5086/api/unidades
Payload com CNPJ já existente: 23.456.789/0001-01
Status: 400 Bad Request
{
  "sucesso": false, "codigoStatus": 400,
  "mensagem": "Já existe uma unidade franqueada cadastrada com o CNPJ '23.456.789/0001-01'."
}
```

Evidência 3: Bloqueio de Venda para Unidade Inativa (Regra de Negócio)

```
POST http://localhost:5086/api/vendas
UnidadeId: 3 (Unidade Barra RJ - Inativa)
Status: 400 Bad Request
{
  "sucesso": false, "codigoStatus": 400,
  "mensagem": "Não é permitido registrar vendas para a unidade 'Café Gourmet - Unidade Barra RJ' pois ela se encontra INATIVA."
}
```

Evidência 4: Bloqueio de Saldo Negativo de Estoque (Regra de Negócio)

```
POST http://localhost:5086/api/estoques/movimentar
Tipo: SaidaVenda | Quantidade solicitada: 99.999 unidades
Status: 400 Bad Request
{
  "sucesso": false, "codigoStatus": 400,
  "mensagem": "Operação não permitida. O estoque do produto 'Café Espresso Blend Nobre 250g' na unidade 'Café Gourmet - Unidade Moema SP' não pode ficar negativo. Saldo disponível: 50, Quantidade solicitada: 99999."
}
```

Evidência 5: Realização de Venda Válida com Atualização Atômica

```
POST http://localhost:5086/api/vendas
Itens: 2 Cafés Blend Nobre (R$ 38,50) + 1 Croissant Francês (R$ 15,00)
Status: 201 Created
{
  "codigoVenda": "VND-20260914-4BD8EE",
  "unidadeNome": "Café Gourmet - Unidade Moema SP",
  "valorTotal": 92.00, "status": "Concluida",
  "itens": [
    { "produtoNome": "Café Espresso Blend Nobre 250g", "quantidade": 2, "subtotal": 77.00 },
    { "produtoNome": "Croissant Francês Folhado Tradicional", "quantidade": 1, "subtotal": 15.00 }
  ]
}
```

Evidência 6: Apuração Periódica de Royalties sobre o Faturamento

```
POST http://localhost:5086/api/royalties/apurar
UnidadeId: 1 | Competência: 09/2026
Status: 200 OK
{
  "unidadeNome": "Café Gourmet - Unidade Moema SP",
  "mesReferencia": 9, "anoReferencia": 2026,
  "faturamentoBase": 624.00, "percentualCobrado": 5.0, "valorRoyalty": 31.20,
  "status": "Pendente", "dataVencimento": "2026-10-10T00:00:00Z"
}
```

Evidência 7: Relatório de Ranking de Unidades por Faturamento

```
GET http://localhost:5086/api/relatorios/ranking-unidades
Status: 200 OK
[
  { "posicao": 1, "codigoUnidade": "FRANQ-SP01", "nomeUnidade": "Café Gourmet - Moema SP", "totalVendas":
4, "totalFaturado": 624.00 },
  { "posicao": 2, "codigoUnidade": "FRANQ-PR01", "nomeUnidade": "Café Gourmet - Batel Curitiba",
"totalVendas": 1, "totalFaturado": 115.50 }
]
```

6. Análise Crítica do Desenvolvimento

Decisões Técnicas e Pontos Fortes

A adoção do SQLite associado ao Entity Framework Core garantiu robustez transacional (ACID) e integridade relacional, ao mesmo tempo em que proporciona portabilidade absoluta para a avaliação pelo professor, dispensando instalações complexas. A arquitetura em camadas com Injeção de Dependência e DTOs permitiu o total isolamento da lógica de negócio e segurança dos dados.

Desafios Superados

Durante a implementação, adaptou-se a projeção de agregações monetárias (decimal) em memória com LINQ to Objects, superando limitações do provider SQLite para o operador Sum, mantendo total conformidade matemática. Além disso, resolveu-se a divergência de validação por regex de versões recentes do OpenAPI no Swagger UI via middleware de normalização.

Melhorias Futuras

Em versões futuras, o sistema poderá incorporar mensageria assíncrona (RabbitMQ) para disparos automáticos de e-mails em alertas de estoque crítico e geração de boletos ou cobrança via QR Code PIX dinâmico diretamente na apuração de royalties.

7. Atendimento aos Critérios de Avaliação

Critério de Avaliação	Peso	Implementação Realizada
Modelagem e banco de dados	15%	13 tabelas relacionais com PKs, FKs, Índices Únicos, Migrations e DbInitializer.
API REST e CRUD obrigatório	20%	CRUD completo de Unidades, Produtos, Fornecedores, Estoques, Vendas, Royalties e Chamados.
Regras de negócio da franquia	20%	Bloqueio de CNPJ duplicado, unidade inativa, estoque negativo e cálculo de royalties.
Autenticação e autorização	10%	JWT Bearer com RBAC (AdminFranqueadora, GestorUnidade, Operador) e senha BCrypt.
Arquitetura e organização do código	15%	Separação Controllers/Services/Repositories/DTOs/Data, SOLID, Async/Await e LINQ.
Consultas, filtros e relatórios	10%	Faturamento, ranking de unidades, produtos mais vendidos, estoque crítico e dashboard.
Documentação e versionamento	10%	Swagger funcional, README completo, suíte .http, relatório e histórico Git no GitHub.
TOTAL	100%	Todos os critérios e módulos obrigatórios atendidos com excelência.

WALLACE F G SILVA — RU: 5146520 — CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER (2026)